To transition from raw memory addresses to managing actual files like data.txt, you can format your Winbond W25Q flash chip with a **FAT filesystem** using the **Adafruit_SPIFlash** and **SdFat (Adafruit Fork)** libraries.
Once formatted, your external flash chip acts identically to a MicroSD card. You can create folders, read/write text files, and append logs without worrying about sectors, pages, or byte addresses.

1. Required Libraries

Install these via the **Arduino Library Manager**:

- **Adafruit SPIFlash**
- **SdFat - Adafruit Fork** (Crucial: ensure you get the Adafruit fork, as it includes the necessary wrappers for external SPI flash chips)

2. The File System Code

This sketch checks if the flash chip is formatted. If it isn't, it formats it into a FAT file system. Then, it tracks the time via your RTC, listens to the Serial Monitor, and appends whatever you type directly into a file named data.txt on the flash chip.

```
#include <SPI.h>
#include <Arduino_GFX_Library.h>
#include <Adafruit_SPIFlash.h>
#include <SdFat.h>
#include "RTC.h"

// --- DISPLAY SETUP ---
Arduino_DataBus *bus = new Arduino_UNOPAR8();
Arduino_GFX *tft = new Arduino_ILI9486(bus, A4, 0, false);

#define BLACK   0x0000
#define WHITE   0xFFFF
#define GREEN   0x07E0
#define CYAN    0x07FF
#define YELLOW  0xFFE0

// --- W25Q FLASH & FATFS SETUP ---
#define FLASH_CS_PIN 10
Adafruit_FlashTransport_SPI flashTransport(FLASH_CS_PIN, &SPI);
Adafruit_SPIFlash flash(&flashTransport);

// Define FatFS filesystem instance using SdFat
FatFileSystem fatfs;

// --- RTC SETUP ---
RTCTime currentTime;
int lastSecond = -1;

void setup() {
```

```
  Serial.begin(115200);

  // 1. Init Display
  tft->begin();
  tft->setRotation(1);
  tft->fillScreen(BLACK);

  // 2. Init RTC
  RTC.begin();
  RTCTime startTime(2, Month::SEPTEMBER, 2026, 14, 30, 00,
DayOfWeek::WEDNESDAY, SaveLight::SAVING_TIME_INACTIVE);
  RTC.setTime(startTime);

  // 3. Init Flash & File System
  tft->setCursor(10, 10);
  tft->setTextColor(WHITE);
  tft->setTextSize(2);

  if (!flash.begin()) {
    tft->println("Flash chip not found!");
    while (1);
  }

  // Check if file system is already formatted. If not, format it.
  if (!fatfs.begin(&flash)) {
    tft->println("Formatting Flash to FAT...");
    if (!flash.fatFSInit(&fatfs)) { // Format entire flash chip
      tft->println("FS Format Failed!");
      while (1);
    }
    tft->println("FS Formatted!");
    delay(1000);
    tft->fillScreen(BLACK);
  }

  tft->setCursor(10, 40);
  tft->setTextColor(YELLOW);
  tft->print("Type in Serial to log to data.txt");
}

void loop() {
  // --- 1. RTC CLOCK UPDATE ---
  RTC.getTime(currentTime);
  if (currentTime.getSeconds() != lastSecond) {
    lastSecond = currentTime.getSeconds();

    char timeStr[10];
    sprintf(timeStr, "%02d:%02d:%02d", currentTime.getHour(),
```

```
currentTime.getMinutes(), currentTime.getSeconds());

    int textWidth = 8 * 2 * 6;
    tft->setCursor(tft->width() - textWidth - 10, 10);
    tft->setTextColor(CYAN, BLACK);
    tft->setTextSize(2);
    tft->print(timeStr);
  }

  // --- 2. FILE WRITING VIA SERIAL ---
  if (Serial.available()) {
    String input = Serial.readStringUntil('\n');
    input.trim();

    if (input.length() > 0) {
      // Open "data.txt" for writing. FILE_WRITE appends data to the
end of the file.
      File dataFile = fatfs.open("data.txt", FILE_WRITE);

      if (dataFile) {
        // Timestamp the log entry
        char timestamp[30];
        sprintf(timestamp, "[%02d:%02d:%02d] ", currentTime.getHour(),
currentTime.getMinutes(), currentTime.getSeconds());

        dataFile.print(timestamp);
        dataFile.println(input); // Write user text
        dataFile.close();        // Always close the file to save
changes to flash

        // Show confirmation on display
        tft->fillRect(10, 80, 450, 40, BLACK);
        tft->setCursor(10, 80);
        tft->setTextColor(GREEN, BLACK);
        tft->print("Logged: ");
        tft->print(input);
      } else {
        Serial.println("Error opening data.txt");
      }
    }
  }
}
```

Key Takeaways for File-Based Flash Usage

- **The First Run:** The code checks fatfs.begin(&flash). If the chip is fresh or unformatted, it automatically formats the entire W25Q chip into a FAT volume. Formatting can take a few

seconds.

- **Standard SD Syntax:** Notice how fatfs.open("data.txt", FILE_WRITE) and dataFile.println() mimic standard Arduino SD card functions. You don't have to calculate memory addresses or erase specific sectors manually anymore; the file system driver handles wear-leveling and block mapping dynamically.
- **Always Close Files:** Always call dataFile.close() after writing. If you omit this, the file pointers won't update properly in the file allocation table, and your data may not persist.